

WuWei app2 モジュール構成と画面設計 方針

三分一信之（三分一技術士事務所）

ver 2.0, 2026-05-22:

Table of Contents

1. 1. 全体分類	2
1.1. 1. resource source : 取得元	2
1.2. 2. resource kind : コンテンツ種別	2
1.3. 3. perspective axis : 視点軸	3
2. 2. 推奨モジュール構成	4
2.1. A. resource 共通層	4
2.1.1. <code>wuwei.resource.js</code>	4
2.2. B. document 層	4
2.2.1. <code>wuwei.document.js</code>	4
2.2.2. <code>wuwei.info.document.js</code>	5
2.2.3. <code>wuwei.edit.document.js</code>	5
2.3. C. video 層	6
2.3.1. <code>wuwei.video.js</code>	6
2.3.2. <code>wuwei.info.video.js</code>	6
2.3.3. <code>wuwei.edit.video.js</code>	6
2.4. D. perspective axis 共通層	7
2.4.1. <code>wuwei.perspective.js</code> または model 内共通関数	7
2.5. E. timeline 層	8
2.5.1. <code>wuwei.timeline.js</code>	8
2.5.2. <code>wuwei.info.timeline.js</code>	8
2.5.3. <code>wuwei.edit.timeline.js</code>	9
2.6. F. contents 層	9
2.6.1. <code>wuwei.contents.js</code>	9
2.6.2. <code>wuwei.info.contents.js</code>	9
2.6.3. <code>wuwei.edit.contents.js</code>	10
2.7. G. resource metadata 層	11
2.7.1. <code>wuwei.info.resourceMetadata.js</code>	11
2.7.2. <code>wuwei.edit.resourceMetadata.js</code>	11
3. 3. 画面設計の分類	12
3.1. 1. Content 本体の画面	12
3.1.1. <code>info-document</code>	12
3.1.2. <code>edit-document</code>	12
3.1.3. <code>info-video</code>	12
3.1.4. <code>edit-video</code>	13
3.1.5. <code>info-image / edit-image</code>	13
3.2. 2. Perspective axis の画面	13
3.2.1. <code>info-timeline</code>	13
3.2.2. <code>edit-timeline</code>	13

3.2.3. <code>info-contents</code>	14
3.2.4. <code>edit-contents</code>	14
3.3. 3. upload 専用画面は fallback にする	14
4. 4. メニュー判定の優先順位	16
4.1. info	16
4.2. edit	16
5. 5. Context resolver の設計	18
6. 6. 代表ノードの操作を分かりやすくする方針	20
7. 7. 推奨する最終モジュール構成	21
8. 8. 移行順序	23
9. 9. 共通 style 定義	24
9.1. 1. 位置づけ	24
9.2. 2. 対象とする style 属性	24
9.3. 3. style profile	24
9.4. 4. <code>wuwei.style.js</code> の責務	25
9.5. 5. <code>wuwei.menu.js</code> との関係	25
9.6. 6. edit / info ペインにおける section 構成	26
9.7. 7. style と perspective の境界	27
10. 10. 対象データモデル定義	28
10.1. 1. <code>current.page</code> と <code>graph</code> の役割	28
10.2. 2. <code>current.page</code> の基本構造	28
10.3. 3. <code>graph</code> の基本構造	29
10.4. 4. Node 共通定義	29
10.4.1. <code>current.page.nodes</code> における Node	29
10.4.2. <code>graph</code> における Node	30
10.5. 5. Node 種別ごとの定義	31
10.6. 6. Content node と resource	31
10.7. 7. Group 共通定義	32
10.8. 8. Group 種別ごとの定義	33
10.9. 9. Representative node	33
10.10. 10. Timeline segment node	34
10.11. 11. Contents PageMarker node	35
10.12. 12. Link 共通定義	36
10.12.1. <code>current.page.links</code> における Link	36
10.12.2. <code>graph</code> における Link	36
10.13. 13. <code>current.page</code> と <code>graph</code> の対応一覧	37
11. 結論	39

以下のように、モジュールの責務と画面設計の対象を分けて整理する。

中心となる考え方は、次です。

upload / remote は「取得元・保管方式」
video / document / image は「コンテンツ種別」
timeline / contents は「コンテンツに対する視点軸」
edit / info は「対象の役割に応じた画面」

Chapter 1. 1. 全体分類

1.1. 1. resource source : 取得元

これは物理属性の違いです。

source	意味	主な属性
defined	ノートキャンバスに定義した要素	Topic, Memo, Generic group (vertical, horizontal, simple), Timeline, Contents, Representative Node
remote	ネット上の URL	URL, title, site label, thumbnail
upload	アップロードされたファイル	remote の属性に加えて、file size, upload date, original filename, storage path, mimeType, preview path
generated	変換生成物	PDF preview, thumbnail, preview image
embedded	将来拡張用	data URL 等

ここで重要なのは、**upload** は UI の主分類にしないことです。 **upload** はあくまで resource metadata です。

1.2. 2. resource kind : コンテンツ種別

これはユーザーが扱う対象の種類です。

```
document
  documentKind: 文書はさらに分けます。
    pdf
    office
    html
    text
video
  videoKind: 映像も分けます。
    youtube
    vimeo
    mp4
    mp3
image
audio
web
other
```

たとえば次のように分類します。

kind	source	sub kind	表示・編集
document	upload	office	document viewer / converted PDF
document	remote	html	iframe / browser
video	remote	youtube	video player
video	upload	mp4	video player
image	upload	image	image viewer

1.3. 3. perspective axis : 視点軸

Timeline / Contents は、単なる video / document の機能ではなく、同じコンテンツに複数の視点を与えるための軸です。

Content (video) node

- └─ Timeline perspective A
- └─ Timeline perspective B

Content (document) node

- └─ Contents perspective C
- └─ Contents perspective D

同じ動画でも、発表構成、技術論点、引用対象など、複数の Timeline 軸を定義できます。

同じ文書でも、章立て、技術論点、コメント対象、引用候補など、複数の Contents 軸を定義できます。

Chapter 2. 2. 推奨モジュール構成

2.1. A. resource 共通層

2.1.1. `wuwei.resource.js`

責務：

```
resource source の判定
resource kind の判定
documentKind / videoKind の判定
拡張子ベースの種類判定
viewer URL の解決
preview URL の解決
metadata の取得
```

代表関数案：

```
wuwei.resource.getSource(resource)
wuwei.resource.getKind(resource)
wuwei.resource.getDocumentKind(resource)
wuwei.resource.getVideoKind(resource)

wuwei.resource.isUploaded(resource)
wuwei.resource.isRemote(resource)

wuwei.resource.isVideo(node)
wuwei.resource.isDocument(node)
wuwei.resource.isImage(node)

wuwei.resource.getOpenUrl(node, option)
wuwei.resource.getViewerUrl(node, option)
wuwei.resource.getPreviewUrl(node, option)
wuwei.resource.getMetadata(node)
```

現在 `menu` / `contents` / `info` / `edit` に分散している判定処理は、最終的にはここへ寄せるのがよいです。

2.2. B. document 層

2.2.1. `wuwei.document.js`

責務：

PDF / Office / HTML / text 文書の扱い
Office Viewer / converted PDF / text viewer / iframe の選択
PageMarker から開く URL の解決

代表関数案：

```
wuwei.document.getDocumentKind(node)
wuwei.document.getDocumentOpenUrl(node, option)
wuwei.document.getPageOpenUrl(node, pageMarker)
wuwei.document.getAnchorOpenUrl(node, pageMarker)
wuwei.document.getPreviewUrl(node)
```

2.2.2. **wuwei.info.document.js**

文書 Content 本体の情報表示。

対象：

PDF
Office
HTML
text

表示内容：

文書ラベル
文書説明
文書種別
URL / original file
preview
open new tab
open new window
resource metadata

2.2.3. **wuwei.edit.document.js**

文書 Content 本体の編集。

編集対象：

label
description
document kind 表示
URL / original path 表示
viewer preference

ここでは PageMarker や Contents 軸は扱いません。それらは `edit.contents` の責務です。

2.3. C. video 層

2.3.1. `wuwei.video.js`

責務：

YouTube / Vimeo / mp4 / mp3 の再生 URL 解決
start / end 指定
埋め込み player URL 生成

2.3.2. `wuwei.info.video.js`

動画 Content 本体の情報表示。

表示内容：

動画ラベル
動画説明
動画種別
URL
player preview
open player
open new tab
open new window
resource metadata

2.3.3. `wuwei.edit.video.js`

動画 Content 本体の編集。

編集対象：

label
description
video kind
URL / original path
player settings
thumbnail / preview

Timeline segment はここでは扱いません。segment は `edit.timeline` の責務です。

2.4. D. perspective axis 共通層

Timeline / Contents は共通概念として **Perspective axis** として扱います。

2.4.1. **wuwei.perspective.js** または **model** 内共通関数

責務：

Perspective axis 共通処理
代表ノード管理
軸線管理
member marker 管理
対象 Content との関連管理

共通属性案：

```
{
  id: "...",
  type: "timeline" | "contents",

  perspectiveKind: "timeline" | "contents",

  targetNodeId: "...",
  representativeNodeId: "...",

  label: "...",
  description: "...",

  orientation: "horizontal" | "vertical",
  length: 600,

  style: {
    axisColor: "...",
    axisWidth: 2
  },

  members: [...]
}
```

代表ノードは、単なる Topic ではなく、

この視点軸を代表する操作入口

として扱います。

2.5. E. timeline 層

2.5.1. `wuwei.timeline.js`

責務：

Timeline perspective の作成・更新・削除
segment marker の管理
start / end / mid segment の管理
動画時間との対応

2.5.2. `wuwei.info.timeline.js`

対象：

Timeline representative
Timeline axis
Timeline segment

表示内容は対象で分けます。

Timeline representative / axis

視点ラベル
視点説明
元動画ラベル
segment 一覧
軸方向
軸長
open player

Timeline segment

元動画ラベル
Timeline ラベル
segment ラベル
start
end
description
player preview

2.5.3. `wuwei.edit.timeline.js`

対象：

```
Timeline representative  
Timeline axis  
Timeline segment
```

編集画面は次の3種類に分けると分かりやすいです。

```
edit-timeline-perspective  
edit-timeline-axis  
edit-timeline-segment
```

ただし実装ファイルは1つでもよく、内部テンプレートだけ分ければ十分です。

2.6. F. contents 層

2.6.1. `wuwei.contents.js`

責務：

```
Contents perspective の作成・更新・削除  
PageMarker の管理  
pageNumber / anchorHref / htmlAnchorHref の管理  
文書 Content との対応
```

2.6.2. `wuwei.info.contents.js`

対象：

```
Contents representative  
Contents axis  
PageMarker
```

Contents representative / axis

```
視点ラベル  
視点説明  
元文書ラベル  
PageMarker 一覧  
軸方向  
軸長
```

ページオフセット
open document

PageMarker

元文書ラベル
Contents 視点ラベル
PageMarker ラベル
pageNumber
anchorHref / htmlAnchorHref
description
preview iframe
open new tab
open new window

2.6.3. **wuwei.edit.contents.js**

対象：

Contents representative
Contents axis
PageMarker

編集画面は次の3種類に整理します。

edit-contents-perspective
edit-contents-axis
edit-contents-page-marker

代表ノードでは、次のセクションを持つ統合画面がよいです。

1. 視点情報
label
description
2. 代表ノードの外観
shape
width
height
fill
line
font
label align / offset
3. 軸属性
direction

```
axis length  
axis width  
axis color  
page offset
```

- 4. 元コンテンツ
 - document label
 - document URL
 - open new tab
 - open new window

2.7. G. resource metadata 層

2.7.1. `wuwei.info.resourceMetadata.js`

upload 由来の物理属性を表示する補助セクションです。

表示項目：

```
source  
original filename  
file size  
uploaded date  
mimeType  
storage path  
preview path  
thumbnail path  
converted PDF path
```

これは `info-document` や `info-video` の中に差し込む補助パネルです。

2.7.2. `wuwei.edit.resourceMetadata.js`

基本的には編集対象ではなく、表示専用でよいと思います。必要な場合だけ、title や rightsなどを編集可能にします。

Chapter 3. 3. 画面設計の分類

3.1. 1. Content 本体の画面

3.1.1. info-document

文書 Content 本体。

対象:

PDF / Office / HTML / text

表示:

文書ラベル

文書説明

viewer

open new tab

open new window

resource metadata

3.1.2. edit-document

文書 Content 本体。

編集:

label

description

document settings

preview / thumbnail

3.1.3. info-video

動画 Content 本体。

対象:

YouTube / Vimeo / mp4 / mp3

表示:

動画ラベル

動画説明

player

open player

open new tab

open new window

3.1.4. edit-video

動画 Content 本体。

編集:

- label
- description
- player settings
- thumbnail

3.1.5. info-image / edit-image

画像 Content 本体。

表示:

- image preview
- resource metadata

編集:

- label
- description
- thumbnail / display settings

3.2. 2. Perspective axis の画面

3.2.1. info-timeline

対象:

- Timeline representative
- Timeline axis
- Timeline segment

表示内容は、代表・軸・segment で切り替えます。

3.2.2. edit-timeline

対象:

- Timeline representative
- Timeline axis

代表ノードの場合は、視点情報 + 代表ノード外観 + 軸属性。segment の場合は、start / end / label / description。

3.2.3. info-contents

対象:

Contents representative

Contents axis

PageMarker

代表ノードの場合は、視点情報 + 元文書 + PageMarker 一覧。PageMarker の場合は、元文書 + PageMarker label + page / anchor + preview。

3.2.4. edit-contents

対象:

Contents representative

Contents axis

PageMarker

代表ノードの場合は、視点情報 + 代表ノード外観 + 軸属性。PageMarker の場合は、label / description / pageNumber / anchorHref / style。

3.3. 3. upload 専用画面は fallback にする

現在の **edit-uploaded** / **info-uploaded** は、将来的には主役から外します。

info-uploaded

edit-uploaded

は、次のような fallback にします。

upload されたが document / video / image と判定できないファイル

または名前を変えて、

info-resource

edit-resource

にする方が分かりやすいです。

Chapter 4. 4. メニュー判定の優先順位

編集・情報表示の振り分けは、次の優先順位がよいです。

4.1. info

1. Timeline representative / axis / segment
-> info-timeline
2. Contents representative / axis / PageMarker
-> info-contents
3. video Content
-> info-video
4. document Content
-> info-document
5. image Content
-> info-image
6. other Content
-> info-resource or info-generic
7. Topic / Memo / Link / generic group
-> existing generic info

4.2. edit

1. Timeline representative / axis / segment
-> edit-timeline
2. Contents representative / axis / PageMarker
-> edit-contents
3. video Content
-> edit-video
4. document Content
-> edit-document
5. image Content
-> edit-image
6. other Content
-> edit-resource or edit-generic

7. Topic / Memo / Link / generic group
-> existing generic edit

この順序が重要です。upload を先に判定すると、upload された mp4 や docx が edit-uploaded に落ちてしまいます。

Chapter 5. 5. Context resolver の設計

`resolveMenuContext()` は、次の情報を返すようにすると保守しやすくなります。

```
{
  target: nodeOrLink,

  targetType: 'node' | 'link' | 'group',

  role:
    'content'
    | 'topic'
    | 'memo'
    | 'genericGroup'
    | 'genericGroupMember'
    | 'perspectiveRepresentative'
    | 'perspectiveAxis'
    | 'perspectiveMarker',

  resourceSource:
    'upload'
    | 'remote'
    | 'generated'
    | 'embedded'
    | null,

  resourceKind:
    'video'
    | 'document'
    | 'image'
    | 'audio'
    | 'other'
    | null,

  documentKind:
    'pdf'
    | 'office'
    | 'html'
    | 'text'
    | null,

  videoKind:
    'youtube'
    | 'vimeo'
    | 'mp4'
    | 'mp3'
    | null,

  perspectiveKind:
    'timeline'
```

```
| 'contents'  
| null,  
  
groupKind:  
  'generic'  
  | 'timeline'  
  | 'contents'  
  | null,  
  
groupShape:  
  'simple'  
  | 'horizontal'  
  | 'vertical'  
  | null  
}
```

メニューは、この context を見て決めます。

Chapter 6. 6. 代表ノードの操作を分かりやすくする方針

Timeline / Contents 代表ノードは自由度が高い分、操作が紛らわしいので、メニュー名を整理した方がよいです。

現在のように、

Edit Style
Axis properties

を並べると、どちらを開くべきか迷います。

おすすめは、代表ノードでは次のようにすることです。

Edit perspective

または日本語なら：

視点軸を編集

その編集画面の中をセクション分けします。

視点情報
代表ノードの外観
軸属性
元コンテンツ

ただし、頻繁に使う場合はショートカットとして次を残してもよいです。

Edit representative
Edit axis

この場合も、同じ [edit-contents](#) / [edit-timeline](#) の中で、開く初期セクションを変えるだけにします。

Chapter 7. 7. 推奨する最終モジュール構成

```
app2/resource/  
  wuwei.resource.js  
  
app2/style/  
  wuwei.style.js  
  wuwei.style.markup.js  
  wuwei.style.css  
  
app2/document/  
  wuwei.document.js  
  
app2/video/  
  wuwei.video.js  
  
app2/timeline/  
  wuwei.timeline.js  
  
app2/contents/  
  wuwei.contents.js  
  
app2/edit/  
  document/  
    edit.document.js  
    edit.document.markup.js  
    edit.document.css  
  video/  
    edit.video.js  
    edit.video.markup.js  
    edit.video.css  
  timeline/  
    edit.timeline.js  
    edit.timeline.markup.js  
    edit.timeline.css  
  contents/  
    edit.contents.js  
    edit.contents.markup.js  
    edit.contents.css  
  image/  
    edit.image.js  
    edit.image.markup.js  
    edit.image.css  
  resource/  
    edit.resource.js  
    edit.resource.markup.js  
  
app2/info/  
  document/
```

```
info.document.js
info.document.markup.js
info.document.css
video/
info.video.js
info.video.markup.js
info.video.css
timeline/
info.timeline.js
info.timeline.markup.js
info.timeline.css
contents/
info.contents.js
info.contents.markup.js
info.contents.css
image/
info.image.js
info.image.markup.js
info.image.css
resource/
info.resource.js
info.resource.markup.js
```

既存との互換を考えるなら、まずは既存の **uploaded** を残しながら、振り分けだけ先に変えるのが安全です。

Chapter 8. 8. 移行順序

一度に全部変えると危険なので、次の順がよいです。

Step 1

判定順を変更

timeline / contents / video / document を uploaded より優先

Step 2

info-document / edit-document を追加

既存 info-uploaded の文書表示機能に移す

Step 3

info-video / edit-video を upload / remote 共通に整理

Step 4

timeline / contents を perspective axis として整理

representative / axis / marker の画面を明確化

Step 5

upload 情報を resource metadata section に分離

Step 6

edit-uploaded / info-uploaded を fallback または edit-resource / info-resource に改名

Chapter 9. 9. 共通 style 定義

9.1. 1. 位置づけ

`wuwei.style.js` は、ノートキャンバス上に表示される対象の見た目を扱う共通モジュールとする。ここでいう style は、取得元が upload か remote か、対象が document か video かといったコンテンツ種別とは独立した、画面上の外観属性である。

style 領域

ノートキャンバス上の見た目を扱う

shape / size / fill / line / font / label align / label offset

content-kind 領域

コンテンツ種別ごとの意味的属性を扱う

document / video / image / resource metadata

perspective 領域

Timeline / Contents の視点軸属性を扱う

axis direction / length / width / color / page offset / segment / PageMarker

したがって、`wuwei.style.js` は画面用共通処理とデータ用共通処理に分けず、style 領域を生成・初期化・読取・反映する共通モジュールとして定義する。

9.2. 2. 対象とする style 属性

分類	属性
基本表示	label, description, visible, opacity
形状	shape, size.width, size.height, size.radius
塗り	style.fill, color
線	style.line.kind, style.line.color, style.line.width, outline
フォント	style.font.family, style.font.size, style.font.color, style.font.align
ラベル配置	labelAlign, labelOffset.x, labelOffset.y, axisPos
リンク	shape, routing.path, style.line, style.font, label, x, y
軸	spine.visible, spine.color, spine.width, length, orientation

style 領域は、通常ノード、代表ノード、Timeline segment、Contents PageMarker、Link、軸の外観に共通して利用する。ただし、軸の意味的属性、例えば Timeline の `timeStart` / `timeEnd`、Contents の `pageOffset` / `pageNumber` / `anchorHref` は、それぞれ `wuwei.timeline.js` / `wuwei.contents.js` の責務である。

9.3. 3. style profile

全対象に同じ項目を表示するのではなく、対象の役割に応じて style profile を使い分ける。

profile	主な対象
normalNode	通常 Topic / Content / Memo
contentNode	document / video / image などの Content 本体
representativeNode	generic group / timeline / contents の代表ノード
timelineSegment	Timeline の start / point / end segment
contentsPageMarker	Contents の PageMarker
link	通常 Link
axis	horizontal / vertical group spine、Timeline axis、Contents axis

9.4. 4. **wuwei.style.js** の責務

```
wuwei.style.getProfile(target, option)
wuwei.style.renderEditSection(target, option)
wuwei.style.renderInfoSection(target, option)
wuwei.style.bindEditSection(root, target, option)
wuwei.style.readEditValues(root, option)
wuwei.style.applyEditValues(target, values, option)
wuwei.style.normalize(target, option)
```

renderEditSection() は、style 領域の HTML を返す。**bindEditSection()** は、align アイコン、color picker、size input、textarea rows などの style 領域の UI を初期化する。**applyEditValues()** は、対象 node / link / group に style 値を反映する。

9.5. 5. **wuwei.menu.js** との関係

wuwei.menu.js は style 処理の本体を持たない。右クリック対象を判定し、どの edit / info domain のどの section を開くかを指定するだけにする。

```
wuwei.edit.open(target, {
  domain: 'contents',
  section: 'style'
});

wuwei.edit.open(target, {
  domain: 'contents',
  section: 'axis'
});
```

例えば Contents 代表ノードでは、同じ **edit-contents** ペインを開き、初期表示 section だけを変える。

メニュー	domain / section	意味
Edit Style	contents / style	代表ノードの外観を編集する

メニュー	domain / section	意味
Axis properties	contents / axis	代表している Contents axis の属性を編集する
Edit perspective	contents / overview	視点情報、代表ノード外観、軸属性、元コンテンツを統合的に編集する

Timeline でも同じ考え方とする。

```
wuwei.edit.open(target, {
  domain: 'timeline',
  section: 'style'
});

wuwei.edit.open(target, {
  domain: 'timeline',
  section: 'axis'
});
```

9.6. 6. edit / info ペインにおける section 構成

```
edit-document
  style section
  document section
  resource metadata section

edit-video
  style section
  video section
  resource metadata section

edit-contents
  style section
  perspective section
  axis section
  PageMarker section
  source document section

edit-timeline
  style section
  perspective section
  axis section
  segment section
  source video section
```

info 側も同様に、対象固有情報と style summary を section として構成する。

```
info-document
```

- document summary
- viewer / preview
- style summary
- resource metadata

- info-contents
 - perspective summary
 - source document
 - PageMarker list / PageMarker detail
 - style summary
 - axis summary

9.7. 7. style と perspective の境界

Timeline / Contents 代表ノードは、代表ノード自身の外観と、代表している視点軸の属性を併せて扱う。自由度が高い一方で操作が紛らわしくなるため、編集画面では section を明確に分ける。

視点情報

- label
- description

代表ノードの外観

- shape
- width
- height
- fill
- line
- font
- label align / offset

軸属性

- direction
- axis length
- axis width
- axis color
- page offset (Contents)
- time range / scale (Timeline)

元コンテンツ

- source content label
- open new tab
- open new window
- open player / viewer

Chapter 10. 10. 対象データモデル定義

10.1. 1. `current.page` と `graph` の役割

WuWei app2 では、`current.page` と `graph` を明確に区別する。

領域	役割
<code>current.page</code>	永続化対象の正本。保存・読込・undo / redo の基準となる。 <code>nodes</code> , <code>links</code> , <code>groups</code> , <code>transform</code> , <code>thumbnail</code> などを保持する。
<code>graph</code>	描画・操作・simulationのためのruntime表現。 <code>current.page</code> から生成され、pseudo link / pseudo group overlay / force simulation 用属性などを含むことがある。

基本方針は次である。

```
current.page
  durable model
  保存対象
  node / link / group の正本

graph
  runtime render model
  描画・ドラッグ・simulation 用
  pseudo link / pseudo overlay / runtime fields を含む
```

`model.setGraphFromCurrentPage()` は `current.page` を `graph` に展開する。`current.page` は正本であり、`graph` は runtime 展開である。real node / real link は原則として `current.page.nodes` / `current.page.links` の同一オブジェクト参照を `graph` 側でも扱うため、`graph` から `current.page` へ変換し直す専用関数は定義しない。ただし、pseudo link や pseudo overlay などの描画補助要素は保存対象にしない。

10.2. 2. `current.page` の基本構造

```
current.page = {
  id: '...',
  pp: 1,
  name: '...',
  description: '...',

  transform: {
    x: 0,
    y: 0,
    scale: 1
  },

  nodes: [],
```

```
links: [],
groups: [],

thumbnail: '...'
};
```

transform は、ノートキャンバス全体の pan / zoom 状態を表す。SVG 描画では、この transform をキャンバスに適用するため、node の座標と画面上の見え方は区別して扱う。

10.3. 3. graph の基本構造

```
graph = {
  nodes: [],
  links: [],
  mode: 'draw' | 'view' | 'simulation',
  ...
};
```

graph には、**current.page.nodes** / **current.page.links** から来る real record に加え、次のような runtime 要素が含まれる。

runtime 要素	説明
pseudo axis link	horizontal / vertical group、Timeline axis、Contents axis を描画するための仮想 Link
pseudo group overlay	simple group の外周や操作入口として使う仮想表示要素
force simulation fields	vx , vy , fx , fy , dragging , changed , filterout など
computed routing	Link path や制御点など、描画時に補助的に使う値

10.4. 4. Node 共通定義

10.4.1. **current.page.nodes** における Node

```
{
  id: '...',
  type: 'Topic' | 'Content' | 'Memo' | 'Segment' | 'PageMarker',
  label: '...', // Memo では使用しない
  description: {
    format: 'asciidoc' | 'plain' | 'plain/text',
    body: '...'
  },

  x: 0,
  y: 0,
  visible: true,
```

```

shape: 'RECTANGLE' | 'CIRCLE' | 'MEMO' | '...',
size: {
  width: 100,
  height: 40,
  radius: 20
},

style: {
  fill: '#ffffff',
  font: {
    family: 'sans-serif',
    size: 12,
    color: '#333333',
    align: 'center'
  },
  line: {
    kind: 'SOLID',
    color: '#808080',
    width: 1
  }
},

audit: {
  createdBy: '...',
  createdAt: '...',
  lastModifiedBy: '...',
  lastModifiedAt: '...'
}
}

```

10.4.2. graph における Node

graph 上の Node は、原則として `current.page.nodes` の real node である。ただし、描画・ドラッグ・simulation のために次の runtime 属性を持つことがある。

```

{
  ...node,
  vx: 0,
  vy: 0,
  fx: null,
  fy: null,
  dragging: false,
  changed: true,
  filterout: false
}

```

これら runtime 属性は、永続データの意味を持たない。保存時には、正本である `current.page` から pseudo node / pseudo link などの runtime 要素を含まない real data を出力する。

10.5. 5. Node 種別ごとの定義

種別	current.page での定義	graph での扱い
Topic	<code>type: 'Topic'</code> 。通常概念ノード。 <code>label</code> , <code>description</code> , <code>shape</code> , <code>size</code> , <code>style</code> , <code>x</code> , <code>y</code> を持つ。	real node として描画される。simulation / drag により runtime 座標や force 属性を持つことがある。
Content	<code>type: 'Content'</code> 。 <code>resource</code> を持つ。 <code>resource</code> により upload / remote、document / video / image 等を判定する。legacy な <code>uri</code> , <code>url</code> , <code>format</code> , <code>download_url</code> は Node 直下に持たせない方針。	real node として描画される。Content 本体の edit / info は resource kind により document / video / image / resource に振り分ける。
Memo	<code>type: 'Memo'</code> 。メモ本文を <code>description.body</code> に持つ。付箋紙風の <code>shape</code> / <code>style.fill</code> を持つ。	real node として描画される。Memo 固有の見た目は style profile で扱う。 Memo は <code>label</code> を持たない。表示本文は <code>description.body</code> として定義する。旧データ又は共通 Node 正規化処理等により Memo に <code>label</code> が存在する場合でも、読み込み時又は保存時に不要項目として無視する。Memo の <code>style.label</code> も v2 データモデル及び保存対象には含めない。
Segment	<code>type: 'Segment'</code> 。Timeline の member node。 <code>groupRef</code> , <code>topicKind: 'timeline-point'</code> , <code>axisRole</code> , <code>mediaStart</code> , <code>mediaEnd</code> , <code>playDuration</code> を持つ。	Timeline axis 上に配置される real node。drag により <code>mediaStart</code> / <code>mediaEnd</code> または <code>axisPos</code> が更新されることがある。
PageMarker	<code>type: 'PageMarker'</code> 。Contents の member node。 <code>groupRef</code> , <code>topicKind: 'contents-page'</code> , <code>documentRef</code> , <code>pageNumber</code> , <code>anchorHref</code> , <code>htmlAnchorHref</code> を持つ。	Contents axis 上に配置される real node。PDF / Office では pageNumber 順、HTML では anchorHref を参照して開く。

10.6. 6. Content node と resource

Content node では、物理属性と意味的分類を `resource` に持たせる。

```
{
  id: '...',
  type: 'Content',
  label: '...',
  description: {...},
  resource: {
    id: '...',
```

```

source: 'upload' | 'remote' | 'generated' | 'embedded',
kind: 'document' | 'video' | 'image' | 'audio' | 'other',
documentKind: 'pdf' | 'office' | 'html' | 'text',
videoKind: 'youtube' | 'vimeo' | 'mp4' | 'mp3',
canonicalUri: '...',
uri: '...',
title: '...',
mimeType: '...',
storage: {
  files: [
    { role: 'original', path: '...', mimeType: '...' },
    { role: 'preview', path: '...' },
    { role: 'thumbnail', path: '...' }
  ]
}
}
}
}

```

mimeType は参考情報として保持・表示するが、処理分岐の主キーにはしない。文書・映像の判定は、原則として拡張子、URL、**resource.kind** / **resource.documentKind** / **resource.videoKind** などに基づく。

10.7. 7. Group 共通定義

Group は **current.page.groups** に保持する。Group 自体は原則として Node ではない。ただし graph 上では、操作や描画のために pseudo node / pseudo link / overlay として現れることがある。

```

{
  id: '...',
  name: '...',
  type: 'simple' | 'horizontal' | 'vertical' | 'timeline' | 'contents',
  groupType: 'axis' | undefined,
  description: {
    format: 'plain' | 'asciidoc',
    body: '...'
  },
  visible: true,
  moveTogether: true,
  orientation: 'auto' | 'horizontal' | 'vertical',
  spine: {
    kind: 'SOLID' | 'DASHED' | '...',
    visible: true,
    color: '#888888',
    width: 4,
    padding: 12,
    paddingTop: 12,
    paddingRight: 12,
    paddingBottom: 12,
    paddingLeft: 12
  },
}

```

```

members: [
  { nodeId: '...', order: 1, role: 'member', value: '', offset: 0 }
],
representativeNodeId: '...',
targetNodeId: '...',
parentGroupId: '',
hierarchical: false,
axis: {...},
length: 480,
audit: {...}
}

```

10.8. 8. Group 種別ごとの定義

種別	<code>current.page</code> での定義	graph での扱い
simple group	<code>current.page.groups[]</code> に <code>type: 'simple'</code> として保持する。 <code>members</code> は real node の <code>nodeId</code> を参照する。代表ノードを持つ場合は <code>representativeNodeId</code> で参照する。	外周 box / overlay は graph 上で描画補助として扱う。group 自体を real node として保存しない。
horizontal group	<code>type: 'horizontal'</code> 。 <code>orientation: 'horizontal'</code> 。 member node の y 座標を中心に揃える group。 spine を持つ。	graph では spine を pseudo link として描画する。 member node と representative node は real node。
vertical group	<code>type: 'vertical'</code> 。 <code>orientation: 'vertical'</code> 。 member node の x 座標を中心に揃える group。 spine を持つ。	graph では spine を pseudo link として描画する。 member node と representative node は real node。
timeline group	<code>type: 'timeline'</code> , <code>groupType: 'axis'</code> 。 video Content を <code>targetNodeId</code> / <code>mediaRef</code> で参照する。 <code>hierarchical: true</code> とし、Content に付属する semantic sub-group として扱う。	graph では Timeline axis を pseudo link として描画する。 Segment node と代表ノードは real node。
contents group	<code>type: 'contents'</code> , <code>groupType: 'axis'</code> 。 document Content を <code>targetNodeId</code> / <code>documentRef</code> で参照する。 <code>hierarchical: true</code> とし、Content に付属する semantic sub-group として扱う。	graph では Contents axis を pseudo link として描画する。 PageMarker node と代表ノードは real node。

10.9. 9. Representative node

Representative node は、Group を代表する real Topic node である。 `current.page.nodes` に保存されるが、`group.members` には含めない。 Group とは `groupRef` / `groupRole` / `representativeOf` / `representativeNodeId` により関連付ける。

```

{
  id: '...',

```

```

type: 'Topic',
label: 'Contents' | 'Timeline' | '...',
description: {...},
groupRef: 'group-id',
groupRole: 'representative',
representativeOf: {
  kind: 'group',
  id: 'group-id'
},
topicKind: 'group-representative' | 'timeline-representative' | 'contents-representative',
representativeType: 'simple' | 'horizontal' | 'vertical' | 'timeline' | 'contents',
axisPos: 0,
shape: 'RECTANGLE',
size: {...},
style: {...},
x: 0,
y: 0
}

```

Representative node の edit / info は、次の2種類の属性を同時に扱う。

代表ノード自身の外観

shape / size / fill / line / font / label align / offset

代表している group / perspective axis の属性

orientation / length / spine / page offset / time range

このため、代表ノードの編集画面では **style** section と **axis** section を分ける。

10.10. 10. Timeline segment node

Timeline segment は **current.page.nodes** に保存される real node であり、Timeline group の **members** から参照される。

```

{
  id: '...',
  type: 'Segment',
  groupRef: 'timeline-group-id',
  topicKind: 'timeline-point',
  axisRole: 'start' | 'point' | 'end',
  mediaStart: 0,
  mediaEnd: 15,
  playDuration: 15,
  label: '00:00',
  description: { format: 'plain/text', body: '...' },
  shape: 'CIRCLE',
  size: { radius: 20 },
}

```

```

style: {...},
x: 0,
y: 0,
axisPos: 0,
visible: true,
audit: {...}
}

```

axisRole	意味
start	Timeline axis の開始端。通常は削除しない。
point	ユーザーが定義する映像 segment。start / end 時刻と説明を持つ。
end	Timeline axis の終了端。通常は削除しない。

10.11. 11. Contents PageMarker node

Contents PageMarker は `current.page.nodes` に保存される real node であり、Contents group の `members` から参照される。

```

{
  id: '...',
  type: 'PageMarker',
  topicKind: 'contents-page',
  groupRef: 'contents-group-id',
  documentRef: 'document-content-node-id',
  axisRole: 'entry',
  pageNumber: 1,
  anchorHref: '#section-id',
  htmlAnchorHref: '#section-id',
  label: '1. Introduction',
  description: { format: 'plain/text', body: '...' },
  shape: 'CIRCLE',
  size: { radius: 18 },
  style: {...},
  x: 0,
  y: 0,
  axisPos: 0,
  visible: true,
  audit: {...}
}

```

PDF / Office 文書では `pageNumber` を主に使用する。HTML 文書では `anchorHref` / `htmlAnchorHref` を使用し、特定 section を開く。Office 文書の PageMarker では、ページ指定のために OpenOffice / LibreOffice で変換した PDF を使用する。EC2 上の通常表示では Office Viewer を使用できるが、PageMarker のページ指定では PDF preview を使う。

10.12. 12. Link 共通定義

10.12.1. `current.page.links` における Link

```
{
  id: '...',
  type: 'Link',
  from: 'source-node-id',
  to: 'target-node-id',
  relation: '...',
  label: '...',
  description: {
    format: 'plain',
    body: '...'
  },
  x: 0,
  y: 0,
  shape: 'NORMAL' | 'HORIZONTAL' | 'VERTICAL' | 'HORIZONTAL2' | 'VERTICAL2',
  visible: true,
  style: {
    font: {
      family: 'sans-serif',
      size: 12,
      color: '#333333',
      align: 'center'
    },
    line: {
      kind: 'SOLID' | 'DASHED' | 'DOTTED',
      color: '#808080',
      width: 1
    }
  },
  routing: {
    path: '...'
  },
  audit: {...}
}
```

`from` と `to` は real node の id を参照する。 `x / y` は主に Link label や制御点の表示位置に使う。 `shape` は link routing の種類を表す。

10.12.2. `graph` における Link

`graph` 上の Link は、 `current.page.links` の real Link に加え、 group / axis を描画するための pseudo Link を含むことがある。

Link 種別	説明
real Link	<code>current.page.links</code> に保存される通常の関係線。ユーザーが作成した Link や、Contents 代表ノードと PageMarker を結ぶ管理対象 Link など。
pseudo group spine	horizontal / vertical group の spine を描画するために graph 上で生成する Link。保存対象ではない。
pseudo timeline axis	Timeline axis を描画するために graph 上で生成する Link。 <code>groupRef</code> , <code>groupType: 'timelineAxis'</code> , <code>linkType: 'timeline-axis'</code> などを持つことがある。保存対象ではない。
pseudo contents axis	Contents axis を描画するために graph 上で生成する Link。 <code>groupRef</code> , <code>groupType: 'contentsAxis'</code> , <code>linkType: 'contents-axis'</code> などを持つことがある。保存対象ではない。

```
{
  id: 'pseudo-link-id',
  type: 'Link',
  pseudo: true,
  groupRef: 'group-id',
  groupType: 'contentsAxis' | 'timelineAxis' | 'horizontal' | 'vertical',
  linkType: 'contents-axis' | 'timeline-axis',
  from: '...',
  to: '...',
  shape: 'NORMAL',
  visible: true,
  style: {...}
}
```

pseudo Link は描画補助であり、`current.page.links` には保存しない。保存時には real Link と pseudo Link を区別する必要がある。

10.13. 13. `current.page` と graph の対応一覧

対象	<code>current.page</code>	graph
Topic / Content / Memo	<code>current.page.nodes</code> に real node として保存する。	<code>graph.nodes</code> に展開して描画・ドラッグ対象にする。
Representative node	<code>current.page.nodes</code> に real Topic として保存し、 <code>group.representativeNodeId</code> で参照する。	<code>graph.nodes</code> に展開して、group / perspective axis の操作入口として扱う。
Segment	<code>current.page.nodes</code> に real Segment として保存し、Timeline group の <code>members</code> で参照する。	<code>graph.nodes</code> に展開し、Timeline axis 上に配置する。
PageMarker	<code>current.page.nodes</code> に real PageMarker として保存し、Contents group の <code>members</code> で参照する。	<code>graph.nodes</code> に展開し、Contents axis 上に配置する。

対象	current.page	graph
Generic group	current.page.groups に保存する。member は nodeId 参照。	pseudo overlay / pseudo spine を生成し、member node と代表ノードを real node として描画する。
Timeline group	current.page.groups に保存する。video Content を targetNodeId / mediaRef で参照する。	pseudo Timeline axis link を生成し、Segment node と代表ノードを軸上に配置する。
Contents group	current.page.groups に保存する。document Content を targetNodeId / documentRef で参照する。	pseudo Contents axis link を生成し、PageMarker node と代表ノードを軸上に配置する。
Real Link	current.page.links に保存する。	graph.links に展開して描画・編集対象にする。
Pseudo Link	保存しない。	group / axis / overlay の描画補助として graph.links にだけ存在する。

Chapter 11. 結論

最も重要な整理は次です。

upload は UI 分類ではなく resource metadata。
video / document / image が Content 本体の UI 分類。
timeline / contents は Content に対する Perspective axis。
代表ノードは Perspective axis の操作入口。

そのため、画面分類は次のようにするのが妥当です。

Content 本体:
edit-document / info-document
edit-video / info-video
edit-image / info-image

視点軸:
edit-timeline / info-timeline
edit-contents / info-contents

物理属性:
resource metadata section

fallback:
edit-resource / info-resource

この整理にすると、upload 文書が **edit-uploaded** に落ちる、upload 動画が **edit-uploaded** に落ちる、PageMarker が **info-uploaded** に展開される、といった混乱を避けられます。